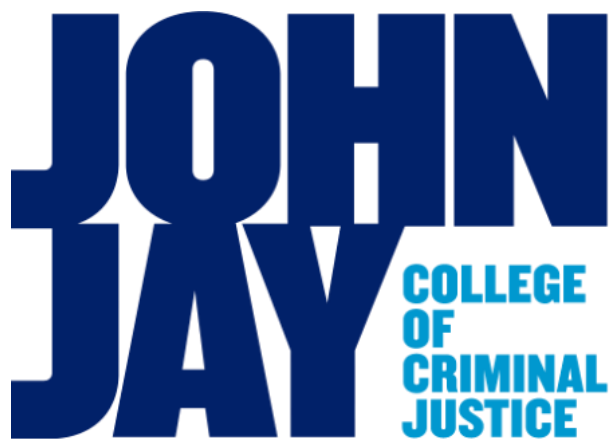


The Security of LLM-Generated Code

By

Christopher Brian Gonzalez Ayala



Thesis submitted in partial fulfilment of the requirements of John Jay College of Criminal Justice, City University of New York, for
Master of Science

Department of Mathematics & Computer Science

John Jay College of Criminal Justice

June 2026

Master Thesis

Department of Mathematics & Computer Science

The Security of LLM-Generated Code

Christopher Brian Gonzalez Ayala

Advisor

Professor Hunter Johnson

First Assessor

Professor Hunter Johnson

Second Assessor

Professor Fatma Najjar



Acknowledgement

First and foremost, I would like to express my sincere gratitude to **Karol V. Mason** for her exemplary leadership as the President of John Jay College of Criminal Justice and for fostering the environment that made my acceptance possible. Her unwavering commitment to student success has been decisive in shaping the academic and personal path I have pursued across both my undergraduate and graduate studies.

Next, I am especially grateful to **Shweta Jain** for her guidance and support throughout my time in the Computer Science & Information Security undergraduate program, as well as the Digital Forensics & Cybersecurity graduate program at John Jay College of Criminal Justice. Her dedication to student development provided me with invaluable knowledge and experiences that have greatly contributed to my academic and professional growth.

I would also like to offer my thanks to **Professor Hunter Johnson** for his mentorship and guidance throughout the course of this thesis project. His ability to provide thoughtful direction, constructive criticism, and continuous encouragement helped me navigate through challenges and refine my ideas at every stage of the project. I am especially appreciative of his dedication, which contributed to the success of this thesis project and my growth as a researcher and student.

I wish to acknowledge the faculty and staff at **John Jay College of Criminal Justice**, especially the professors within the Computer Science department, whose knowledge, instruction, and mentorship have profoundly shaped my academic journey.

I would also like to acknowledge my colleagues, whose intelligence, dedication, and professional experiences have served as a source of inspiration. Observing their commitment to excellence has encouraged me to continually strive for growth and to perform at my highest potential.

Finally, I am deeply thankful to my friends and family for their unwavering moral support, encouragement, and belief in me throughout this journey.

It is my hope that this work serves as a source of insight and inspiration for all who read it.

Christopher Brian Gonzalez Ayala

Abstract

The rapid adoption of Large Language Models (LLMs) in software development has transformed coding practices by enabling automated code generation, completion, and optimization. Despite these advantages, concerns persist regarding the security and reliability of LLM-generated code. This study presents a comprehensive evaluation of both the functional correctness and security of code produced by three prominent LLMs as of early 2026. A total of 4,800 code snippets were generated using 100 security-focused programming prompts derived from the OWASP Top 10:2025, translated across eight natural languages and two phrasing styles (literal and natural developer-oriented prompts). To assess performance, a multi-stage experimental framework was developed, incorporating automated code generation, syntax auditing, and cross-model evaluation. Each code sample was scored for correctness and security using LLM-based evaluators, with additional classification of vulnerabilities based on OWASP risk categories and Common Weakness Enumeration (CWE) root causes. The findings reveal a consistent and significant “security-correctness gap,” where LLMs frequently produce functionally correct code that contains exploitable vulnerabilities. Statistical analysis indicates only a moderate correlation between correctness and security, reinforcing that functional success does not imply safety.

Keywords: Large Language Models (LLMs), Cybersecurity, LLM-Generated Code, Prompt Engineering, OWASP, CWE, Security-Correctness Gap

Table of Contents:

Chapter Name	Page No.
1. Introduction	1
1.1 Overview of AI and LLM Technology	2
1.2 Motivation Behind Assessing the Security of LLM Code	4
1.3 Scope and Limitations	6
1.4 Research Objectives	7
2. Literature Review	9
2.1 LLMs in Software Engineering	10
2.2 Cybersecurity Concerns in LLM Output	11
2.3 Prompt Engineering and Linguistics	14
2.4 Other Previous Work	18
3. Methodology	21
3.1 Experimental Setup	22
3.2 Dataset Generation	25
3.3 Evaluation Metrics	29
4. Data Presentation & Results	32
4.1 Overall Performance	33
4.2 Linguistic Impact	38
4.3 Phrasing Impact	40
4.4 Other Findings	42
5. Discussion	46
6. Conclusion & Future Work	50
References	54
GitHub Link	57

List of Figures

Figure Name.	Page No.
Figure 1 – El Ajedrecista by Leonardo Torres Quevedo	3
Figure 2 – Motivation for using and not using AI programming assistants	5
Figure 3 – LLM applications	11
Figure 4 – APE vs. human prompts: zero-shot accuracy on 24 tasks	15
Figure 5 – Heatmap of LLM performance across programming languages	17
Figure 6 – Folder hierarchy system for LLM-code generation	25
Figure 7 – System message for LLM code-evaluators	30
Figure 8 – Basic statistics for overall correctness and security scores	33
Figure 9 – Average correctness and security scores by the LLM evaluator	34
Figure 10 – OWASP security risk distribution	35
Figure 11 – OWASP security risk distribution by the LLM generator	36
Figure 12 – Root causes distribution by the LLM generator	37
Figure 13 – Average correctness scores by language and LLM generator	38
Figure 14 – Average security scores by language and LLM generator	38
Figure 15 – Heatmap of OWASP security risks by natural language	39
Figure 16 – Average correctness and security scores by phrasing and LLM generator	40
Figure 17 – Average correctness scores by natural language, phrasing, and LLM generator	41
Figure 18 – Average security scores by natural language, phrasing, and LLM generator	42
Figure 19 – Pearson correlation coefficient between code correctness and security	43
Figure 20 – Self-bias analysis for correctness and security scores	44
Figure 21 – Correctness and security scores by the LLM generator	45

Chapter 1 - Introduction

Introduction

1.1 Overview of AI and LLM Technology

In 1726, Johnathan Swift published the novel “Gulliver’s Travels”, which presented the idea of a large mechanical machine, The Engine, that could be used to assist scholars in generating new ideas and writing. It was proposed that scholars would turn the handles on The Engine, which would then rotate wooden blocks inscribed with words to create new ideas by combining the words in different arrangements [1]. It can be argued that Swift’s novel was an early depiction of automatic text generation, which the world associates with Artificial Intelligence 300 years later. Artificial Intelligence, or AI for short, has become a core technology in modern society. While Swift’s idea of The Engine was strictly limited to text generation, AI is capable of translation services, media generation, automation, predictive analysis, coding, anomaly detection, and more.

Most people would believe that AI technology is exclusive to the 21st century, but the first fully automated machine was invented in 1914. Spanish engineer Leonardo Torres Quevedo built a chess-playing machine known as El Ajedrecista, which required no human intervention after assembly [1]. This invention is a parallel to The Engine in that both embodied automatic decision-making and served as the early origins of AI. The term “artificial intelligence” was not created until 1955, when John McCarthy of Dartmouth College, Marvin L. Minsky of Harvard University, Nathaniel Rochester from IBM, and Claude E. Shannon from Bell Telephone Laboratories published “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence” in a workshop. The aim was to proceed on the basis that every aspect of learning or any other feature of intelligence can be so precisely described that a machine can be made to simulate it [2].

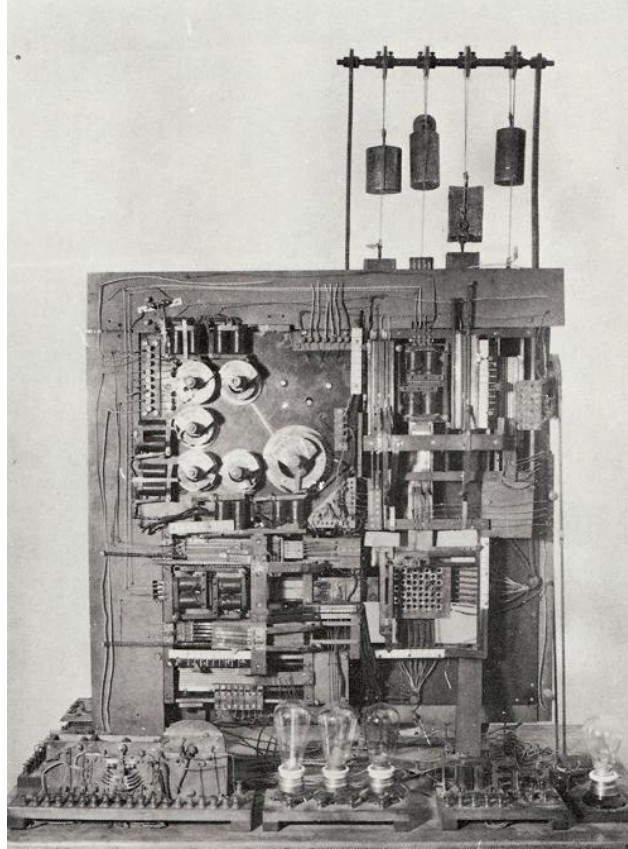


Figure 1: El Ajedrecista by Leonardo Torres Quevedo. Adapted from [3]

These critical moments in technological history set the stage for the world to witness the foundational model for the generative AI era. In June 2020, the Artificial Intelligence company OpenAI introduced the Generative Pre-trained Transformer 3 (GPT-3), a large language model that generates text, engages in conversations, writes code, translates languages, and generates creative writing [1]. In other words, OpenAI exceeded the machine foretold in Swift's novel by eliminating the need for manual labor and creating a technology capable of performing tasks rapidly. Instead of requiring scholars to turn handles, GPT-3 was the successor to prior transformers from OpenAI (GPT-1 and GPT-2) and trained on vast amounts of internet data to become a Large Language Model (LLM). This suggests that all LLMs are a form of AI, but not

all AI models are LLMs. To avoid confusion, LLMs have been defined as computer programs that are trained to recognize and interpret human language or other types of data [4]. Ultimately, GPT-3 was a turning point in the advancement of automated technology, reshaping what machines and people can achieve, like coding.

1.2 Motivation Behind Assessing the Security of LLM-Generated Code

In tandem with the evolution of technology, the education system is expected to undergo significant changes for the foreseeable future. For example, technology skills, such as problem-solving, are being introduced to Asian-Pacific students earlier in the school curriculum. Specifically, these students are exposed to coding and software development through game playing, game creation, and the development of applications that do not require computer interfaces [5]. The shift to incorporate computer science into the education system for young students began around the mid-2010s. The United States would eventually acknowledge the growing subject by attempting to adapt computer science classes into school curricula. Unfortunately, the U.S.’s decentralized school system creates challenges for enacting nationwide reform. This is because a significant number of schools have yet to update their standards, and students of different ethnic and socioeconomic backgrounds demonstrate disparities in their knowledge of computers [6]. Despite this, the world has recognized that programming skills are becoming required for life in the 21st century.

For years, coding has been a skill that requires patience, dedication, and a reliable problem-solving mindset. In fact, emerging programmers have typically found the initial learning curve of coding frustrating, but modern technology has the potential to accelerate productivity, correct errors, and enable programmers to focus their energy on other tasks. In recent times, some have referred to the collaboration of programmers and automated services as “vibe coding”, which has

demonstrated real-world uses. A 2024 survey reveals that 86% of GitHub users find it very important to use AI programming assistants for autocompletion, 76% to finish programming tasks faster, and 68% to skip the need to go online to find specific code snippets, programming syntax, or API calls [7]. In other words, modern programmers have come to rely on AI (and LLMs) out of convenience, reducing the frustration associated with coding.

Despite the potential benefits, this raises the need to carefully assess the quality and security of generated code to ensure that these powerful tools do not inadvertently introduce vulnerabilities or threats. The survey also revealed that 54% of GitHub users do not use programming assistants because the tools write code that doesn't meet the functional or non-functional requirements needed, while 48% claim difficulty in controlling code generation tools to get the desired code, and 38% expressed spending too much time debugging or modifying code written by code generation tools [7]. These responses suggest that automated programming assistance, such as ChatGPT, can introduce undesired complexity and issues to the correctness and security of the requested code, raising questions about whether AI and LLMs can be trusted to create programs that are secure against known vulnerabilities and attacks.

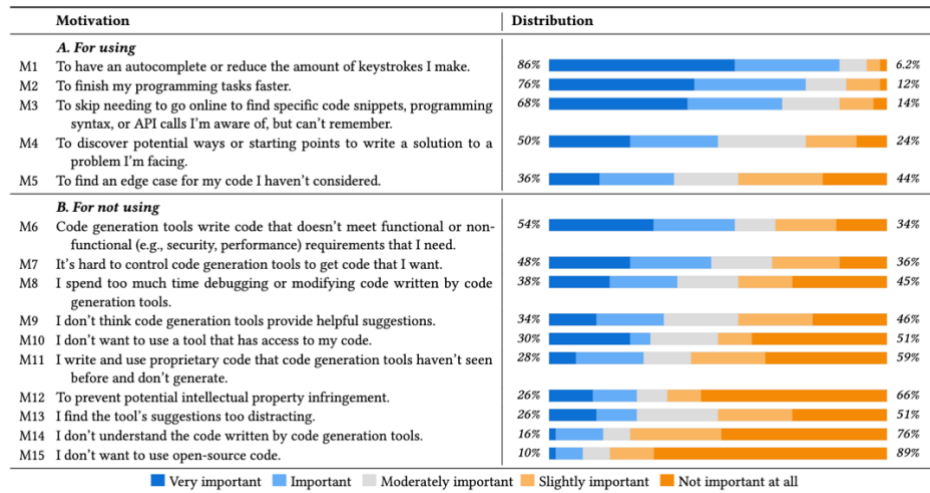


Figure 2: Motivation for using and not using AI programming assistants. Adapted from [7]

This thesis paper seeks to directly address one critical concern: the security of LLM-generated code. By evaluating the potential risks and vulnerabilities in code produced by LLM tools, this research will clarify their practical implications for cybersecurity and chart a path toward safer, more reliable software development.

1.3 Scope and Limitations

This study assesses the correctness and security of LLM-generated code by utilizing Application Programming Interface (API) keys for some of the most prevalent LLM services of 2026. An API can be understood as a set of rules that enable software applications to communicate with each other and exchange data [8]. API keys are randomly generated strings of alphanumeric characters. APIs use them to authenticate the applications making calls, and enable the API server to identify which software, applications, and users are sending a call, as each key is associated with a specific API client [8]. Due to time constraints, only three LLM platforms were utilized for the study. However, each platform was responsible for generating 1,600 programs, totaling 4,800.

In generating the 1,600 programs, the LLM platforms not only relied on prompts fed into their APIs but also accounted for the global nature of coding practices. To ensure inclusivity and broader applicability, these prompts were provided across different natural languages, reflecting diverse programming approaches and linguistic backgrounds from around the world. In 2025, it was discovered that China leads the world in having programmers, 7 million, followed by India, the United States, Japan, and Germany [9]. Moreover, the prompts were expressed in two forms: a literal translation and a translation intended to imitate natural developer phrasing. The purpose of this is to check that the effects measured are not an accident of a specific translation. The prompts were translated through an automated program and may contain some errors.

Finally, it is worth noting that the study utilized the most recent models for the three LLM platforms as of February 2026 (GPT-5.2-Codex, Claude Sonnet 4.6, Gemini 3 Flash), with one exception. LLMs are constantly improving; the best model of a given service as of the study will not persist as the best model as time progresses. Readers who intend to replicate the project must ensure to use the same models, which will be provided further on. Additionally, staying attuned to ongoing advancements in LLM technology is crucial for maintaining the study's relevance and accuracy.

1.4 Research Objectives

The primary aim of this research is to evaluate the security and functional integrity of code generated by LLMs when subjected to linguistic and contextual variables. To achieve this, the study is guided by the following research objectives:

1. Quantify the “Security-Correctness Gap” in generated code

This objective will determine the correlation between a model's ability to produce functional code and its ability to follow secure coding standards. Thus, measuring correctness and security scores across all 4,800 code snippets will answer whether functional proficiency potentially serves as a false indicator of code safety.

2. Investigate the impact of prompt phrasing on the security of generated outputs

This objective aims to observe how variations in prompt phrasing, literal translations against natural developer phrasing, affect the likelihood of a model's code being subject to known vulnerabilities. It may be found that developer-centric language results in more secure code or the contrary.

3. Assess the influence of natural language diversity on code security

This objective investigates whether LLMs demonstrate a bias in code generation by evaluating prompts translated across several distinct (and popular) natural languages. The results will determine whether certain languages produce a higher frequency of security vulnerabilities in the corresponding code compared to English-language baselines.

4. Evaluate the efficacy and potential bias of LLMs as automated security auditors

This objective focuses on developing a cross-model evaluation framework and examines how LLM platforms perform when tasked with grading their own code and the code of other models, quantifying the extent of self-bias and evaluation rigor.

5. Identify the root causes of recurring security failures in LLM-generated code

This objective performs a qualitative analysis on specific architectural weaknesses that LLM models consistently fail to address in their generated code without explicit security prompting. These architectural weaknesses are then categorized by predetermined root causes and critical web application security risks.

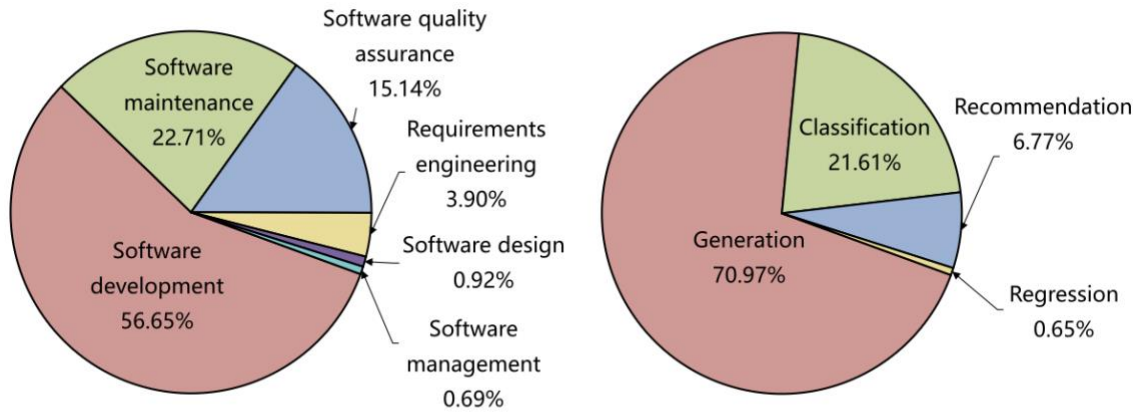
Chapter 2 - Literature Review

Literature Review

2.1 LLMs in Software Engineering

LLMs are a relatively recent technology, debuting in the 20th and 21st centuries. As a reminder, LLMs are computer programs trained to recognize and interpret human language or other data [4]. Within a short period, LLMs have already been utilized in real-world applications, such as software engineering. To clarify, software engineering refers to a systematic approach to the analysis, design, development, testing, implementation, maintenance, and reengineering of computer software [10]. In this context, a software engineer is primarily responsible for staying up to date with software and technological advancements relevant to the software's functionality and security. As a result, software engineering can be considered challenging due to the need for constant learning, problem-solving, resilience, and adaptability.

Fortunately, LLMs have been utilized to simplify the tasks of a software engineer. A 2024 study reveals that LLMs have been applicable in activities such as requirements engineering, software design, software development, software quality assurance, software maintenance, and software management. Of the six activities, LLMs are particularly prevalent in software development, which may involve tasks such as code generation, code completion, code summarization, code search, and others [11]. This suggests that LLMs, and by extension AI, are seen as practical for software engineers for coding-related tasks, thus reinforcing the need to determine whether AI can be trusted to create usable code that is correct, reliable, and secure against prevalent vulnerabilities and attacks.



(a) Distribution of LLM usages in SE activities. (b) Problem classification based on collected studies.

Figure 3: LLM applications. Adapted from [11]

Beyond coding, LLMs have also been utilized in tasks such as streamlined extended reality (XR) prototyping, user story quality analysis, and Unified Modeling Language (UML) modeling [12]. These activities share a common focus on enhancing the efficiency, accuracy, and clarity of the software development lifecycle (SDLC) through abstraction, automation, and improved communication. For example, streamlined XR prototyping enables users to create, update, and delete multidimensional objects without requiring code. In fact, XR tools such as AtomXR allow users to complete these tasks through natural language, eye-gaze, and touch interactions while inside a headset [13]. LLM technology is constantly evolving, so it is essential to address the current issues with this technology, especially in the context of coding and cybersecurity.

2.2 Cybersecurity Concerns in LLM Output

As of the writing of this paper, there is no singular universal definition of the term “cybersecurity”. It ultimately depends on who is describing the term. The government’s definition may have a different focus compared to the definition created by technical experts or

academic researchers. For the purposes of this paper, cybersecurity will be understood by the definition of the National Institute of Standards and Technology, or NIST for short. NIST defines cybersecurity as “The ability to protect or defend the use of cyberspace from cyber attacks” [14]. Moreover, cyberspace is defined as “A global domain within the information environment consisting of an interdependent network of information systems infrastructures, including the Internet, telecommunications networks, computer systems, and embedded processors and controllers” [14]. Finally, a cyber attack is the “targeting (of) an enterprise’s use of cyberspace for the purpose of disrupting, disabling, destroying, or maliciously controlling a computing environment/infrastructure, or destroying the integrity of the data or stealing controlled information” [14].

Given this context, it is crucial to understand the extent of (cyber)security in output created by LLMs. While the amount of training these tools receive can influence the quality of their output, it has been found that a prominent deciding factor in an LLM's output is the input given by the user. A 2025 study found that LLMs struggle with rejecting underspecified inputs, thus leading to the possibility of demonstrating unreliable behavior, such as hallucinated responses [15]. Hallucinated responses are a phenomenon in which an LLM perceives patterns or objects that are nonexistent or imperceptible to human observers, creating outputs that are nonsensical or altogether inaccurate [16]. The significance of this study is how LLMs fail to properly handle obscure inputs. Hence, users who use LLMs and unknowingly provide input that is vague will be subject to output that can pose issues and risks, especially when the output is generated code. Failing to recognize hallucinated code and implementing it in software applications creates prodigious cybersecurity concerns.

While NIST provides comprehensive, high-level cybersecurity frameworks and standards for risk management, its sources offer a broad view of organizational cybersecurity. Essentially, NIST defines what people should do rather than elaborate on how to achieve cybersecurity. This focus stems from their target audience, which includes government agencies, regulated industries, and enterprises that must align with compliance standards, not software engineers or developers [17]. To bridge the gap between high-level guidelines and practical implementation, software engineers seeking actionable advice turn to the Open Worldwide Application Security Project (OWASP) for guidance. OWASP is an open community dedicated to conceiving, developing, acquiring, operating, and maintaining applications that can be trusted, providing resources for improving application security [18]. In particular, OWASP is widely recognized for its OWASP Top 10, the reference standard for the most critical web application security risks.

Similarly, software engineers can identify a wide range of software and hardware weaknesses by turning to the Common Weakness Enumeration (CWE). CWE is a community that documents discovered software and hardware weaknesses that could have security ramifications. A weakness is defined as a condition in a software, firmware, hardware, or service component that could contribute to vulnerabilities [19]. While OWASP provides a prioritized list of web application risks for awareness, CWE is a detailed, technical dictionary of root-cause software weaknesses. These weaknesses are named using a format that uses a unique identifier (e.g., CWE-79) followed by a descriptive title for the weakness (e.g., Improper Neutralization of Input During Web Page Generation) [19]. CWE also offers the Top 25 Most Dangerous Software Weaknesses for the year 2025, but software engineers may also choose to view CWEs by software development, hardware development, or other specified criteria that best suit their purposes.

This interconnected landscape between cybersecurity standards and LLM system behavior sets the stage for a deeper discussion on how prompt engineering and linguistics influence the reliability and safety of generated outputs.

2.3 Prompt Engineering and Linguistics

Prompt engineering is the process of designing input inquiries (prompts) to obtain a desired response from LLMs [20]. As mentioned previously, LLMs have demonstrated challenges in rejecting inputs that are underspecified or vague, resulting in hallucinated responses that are unreliable to the user. Of course, the technology is recent, thus subject to occasional errors and mistakes. While one feasible solution is the continuation of training that allows the technology to properly handle poor input, users should incorporate prompt engineering when collaborating with LLMs and other AI. In fact, all users should be encouraged to practice prompt engineering because it supports the development of more sophisticated and effective LLM applications, helps developers better understand the behavior of LLMs, and enables the facilitation of LLMs in a manner that should bring the most satisfaction to users [20].

Several prompt engineering techniques have been discovered and proven effective by previous work. One example of an advanced technique is automatic instruction generation, which refers to using technology to automatically generate prompts that will be given to an LLM [20]. The concept behind this technique is that automatically generated instructions perform better compared to instructions created by people. This was affirmed in a 2023 study, in which an Automatic Prompt Engineer (APE), InstructGPT, achieved equal or better results than human performance in a zero-shot test [21].

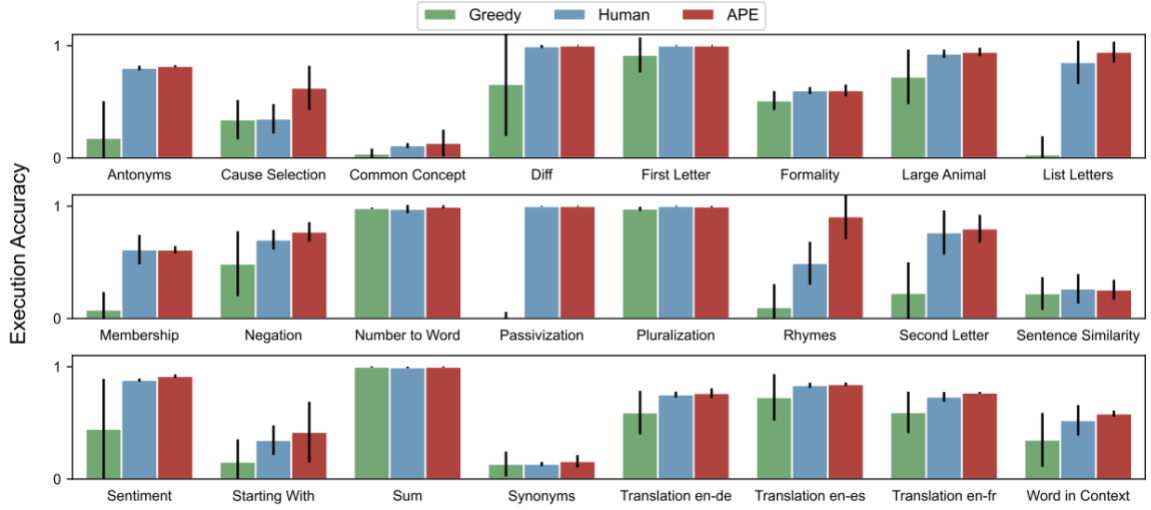


Figure 4: APE vs. human prompts: zero-shot accuracy on 24 tasks. Adapted from [21]

While human-made input demonstrated exceptional results in the figure above, instructions created by APEs are a viable option when interacting with LLM tools. Another advanced technique that has been proven effective is program synthesis. This technique is similar to automatic instruction generation but has some distinct concepts. Automatic instruction generation refers to the use of technology to generate instructions. In contrast, program synthesis, particularly in the context of prompt engineering for computer programs, is a field of research that focuses on automatically generating programs from advanced specifications in natural language descriptions [20].

Natural language, in this context, refers to typical human language (e.g., English), providing an optimal condition for LLMs to generate code. For example, in 2021, it was discovered that a program can be specified by a short natural language description, and when this is combined with human feedback, it results in a 50% reduction in errors [22]. Thus, LLMs can generate working programs from natural language descriptions, with their performance improving

significantly with scale and human feedback. Nevertheless, users must remember that the technology is not perfect and remains subject to occasional errors.

It is also important to consider that coding is performed across the globe in differing linguistic environments. Although most programming code is written in English, it is possible for comments, documentation, and function names to be expressed in the native language of the country the program originates from (e.g., Chinese, Spanish, Hindi, etc.). In turn, this raises the question of whether linguistics has an impact on LLM-generated code. Unfortunately, research on this topic is limited to the quality of code across programming languages. Like natural language, there are numerous programming languages that exist, as different problems require different tools to solve them. One example of a programming language is C++, a general-purpose programming language that supports data abstraction, object-oriented programming, and generic programming [23]. A significant number of programming languages share similarities in their syntax, thus making the process of learning other programming languages familiar.

The most common programming language is Python, because it offers a large set of built-in libraries, making coding easier. These built-in libraries can be applied across computer science domains, including data science, machine learning, deep learning, image processing, cloud computing, and more [24]. It is worth mentioning that LLMs are the product of technology that has undergone both machine learning and deep learning, strengthening the popularity of the Python programming language over others. Examples of other (popular) programming languages include HTML, JavaScript, Ruby, Structured Query Language (SQL), C, and Swift. Determining whether programming languages have an impact on LLM-generated code works similarly to determining whether linguistics has an impact on LLM-generated code because, in both cases, it

is investigated how the structure and rules of a language shape the outputs produced by large language models.

One of the primary reasons LLMs are used for code generation is that the technology has been trained on a vast quantity of code datasets available on the Internet. As a result, developers can use LLMs to boost efficiency in tasks such as code generation [25]. The success of code generation for a given programming language, such as Swift, will depend on the number of Swift datasets available to train the LLM. Given this context, programming languages can be categorized into two main types: Low-Resource Programming Languages (LRPLs) and High-Resource Programming Languages (HRPLs). LRPLs are underrepresented programming languages on the Internet, resulting in weak LLM training data, which leads to lower performance. In contrast, HRPLs are common programming languages on the Internet, resulting in ample training data for LLMs, leading to higher performance [25].

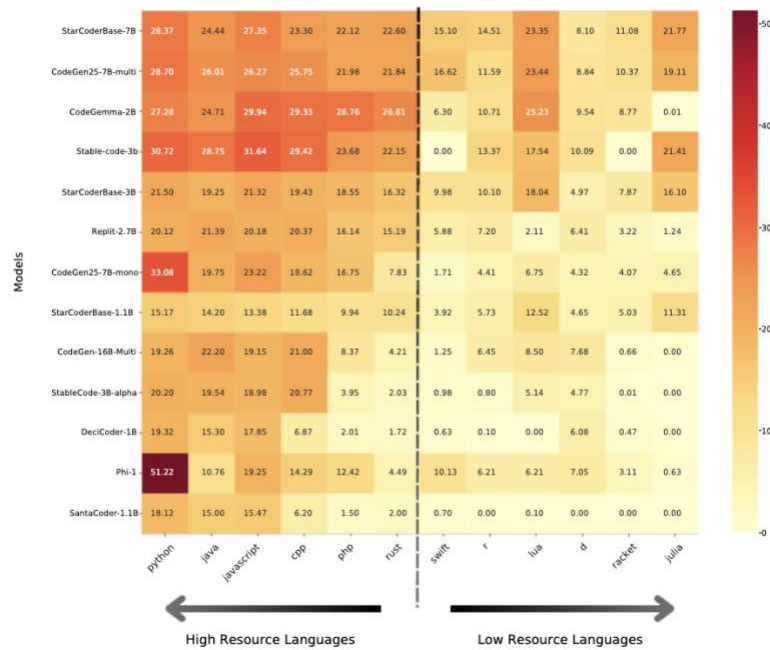


Figure 5: Heatmap of LLM performance across programming languages. Adapted from [25]

From the visual, it can be observed that examples of LRPLs include Lua, Racket, and Julia, whereas examples of HRPLs are Python, Java, and JavaScript. Moreover, the heatmap illustrates that model performance (represented by color intensity) is higher when working with HRPLs compared to LRPLs. This finding should inspire software and LLM developers to increase the representation of LRPLs on the Internet. In turn, this will result in bigger training datasets for enhancing the reliability and overall effectiveness of LLMs in code generation across all programming languages.

2.4 Other Previous Work

Cassano et al. (2022) [26] address the lack of multilingual benchmarks for evaluating LLMs on code generation. While models like Codex are often trained on multiple languages, they are predominantly evaluated in common languages such as Python. Their system, MultiPL-E, solves this by using a set of "little compilers" to translate existing benchmarks into 18 additional programming languages, including C++, Rust, and even LRPLs like R. MultiPL-E translates function signatures, unit tests, and natural language descriptions while standardizing evaluation through a containerized sandbox. Their work demonstrated that a scalable, compiler-based translation approach is a viable and rigorous strategy to benchmark LLMs across the diverse programming landscape.

Yan et al. (2024) [27] introduce a comprehensive framework (CodeScope) designed to address the limitations of existing coding benchmarks, like MultiPL-E, and rely on surface-level text matching rather than functional correctness. CodeScope evaluates LLMs across 43 programming languages and eight diverse tasks, which can be categorized into code understanding (summarization, smell, review, and automated testing) and code generation (program synthesis, translation, repair, and optimization). CodeScope also analyzes model performance across three

specific dimensions: length, difficulty, and efficiency. The authors demonstrate that while some models excel in specific tasks, significant challenges remain in consistently handling multilingual environments and complex data flows, providing a more rigorous standard for future code-related LLM research.

Wang et al. (2025) [28] present MultiOOP, a benchmark that particularly focuses on evaluating tasks in relation to object-oriented programming (OOP) across six programming languages (Python, PHP, C++, C#, Java, JavaScript). Although MultiOOP uses fewer languages, each was given a total of 267 tasks that go beyond function-level generation to evaluate core OOP principles, such as inheritance and encapsulation. Building on this, the authors conducted their study using 14 mainstream LLMs, revealing a substantial performance degradation of up to 65.6% compared to other benchmarks, such as HumanEval. Ultimately, these findings suggest that a model's OOP proficiency is driven more by the diversity of its training data than the popularity of a specific programming language.

Li et al. (2025) [29] share a specialized evaluation framework designed to address overlooked security risks in LLM-generated code. This was achieved by creating a dataset that encompasses diverse development scenarios and critical vulnerability types, including injection flaws, memory safety violations, and insecure configurations. The framework is a hybrid automated evaluation pipeline that combines Static Application Security Testing (SAST) tools with LLM-based judging to detect weaknesses that functional benchmarks may miss. Through testing, the study reveals significant shortcomings in the security of LLM outputs, highlighting the need for security-focused training and providing a standardized methodology for developing more robust, vulnerability-free code generation models.

Peng et al. (2025) [30] developed CWEVAL, a novel evaluation framework and benchmark designed to assess both the functional correctness and security of code generated by LLMs. Unlike benchmarks that contain vague task specifications, CWEVAL utilizes outcome-driven test oracles and human-verified coding scenarios to provide high accuracy and full reproducibility. The benchmark covers 119 security-critical tasks across 31 CWE types and five programming languages. Through empirical security evaluation of GPT-4o, Claude 3.5, Gemini 1.5 Flash, and Llama 3.1 70B Instruct, the authors demonstrate that models often generate code that is functionally correct but contains serious security vulnerabilities, thus revealing a performance drop of approximately 30% when security is required alongside functionality.

Rojpaisarnkit et al. (2025) [31] explore the relationship between a developer's English language proficiency and the quality of code generated by LLMs. This was achieved by rephrasing 164 programming tasks in the HumanEval dataset across the six CEFR proficiency levels (beginner, elementary, intermediate, upper intermediate, advanced, and proficient). The researchers examined how linguistic variations impact code correctness and complexity across various LLM models, including GPT-4o, Gemini 2.5 Pro, and Claude Sonnet 4. Their findings reveal that while LLMs typically default to an intermediate (B2) linguistic level in their output, the accuracy of the generated code significantly improves when provided with higher-proficiency prompts. This research suggests that beyond prompt engineering techniques, the natural language proficiency of the user is a critical factor in the reliability of LLM-assisted programming, suggesting a potential barrier for non-native English speakers in achieving optimal performance from these tools.

Chapter 3 - Methodology

Methodology

3.1 Experimental Setup

This study aims to determine the security and functional integrity of code generated by LLMs when subjected to linguistic and contextual variables. To begin, three of the most prominent Artificial Intelligence (and by extension LLM) companies were utilized: OpenAI, Anthropic, and Google. As mentioned previously, OpenAI introduced the Generative Pre-trained Transformer (GPT), which is commonly known as ChatGPT. In comparison, Anthropic is an AI safety and research company that developed the Claude LLM in March 2023. Finally, Google is a multinational technology corporation that focuses on software, search engine technology, and artificial intelligence in recent years. While predating the former companies, Google's LLM, Gemini, has the most recent release date of the three (December 2023).

A total of 100 coding prompts were created for this study and initially stored in a spreadsheet named "AI_Generated_Code_Analysis.csv". The prompts were completed based on the most critical security risks to web applications seen in the OWASP Top 10:2025 list provided below:

1. [A01:2025 - Broken Access Control](#)
2. [A02:2025 - Security Misconfiguration](#)
3. [A03:2025 - Software Supply Chain Failures](#)
4. [A04:2025 - Cryptographic Failures](#)
5. [A05:2025 - Injection](#)
6. [A06:2025 - Insecure Design](#)
7. [A07:2025 - Authentication Failures](#)
8. [A08:2025 - Software or Data Integrity Failures](#)
9. [A09:2025 - Security Logging and Alerting Failures](#)
10. [A10:2025 - Mishandling of Exceptional Conditions](#)

Each of the security risks in the OWASP Top 10:2025 list was represented by 10 unique prompts to ensure an equal and balanced coverage. Readers of this document are encouraged to learn about these web application risks and understand why they are critical to cybersecurity. In addition to mapping each coding prompt to an OWASP security risk, prompts are also linked with one potential vulnerability that can be exploited if the resulting program is poorly made. A significant amount of the given vulnerabilities come from the CWE, while others are vulnerability concerns specific to the prompt. Readers are also encouraged to explore these continuing vulnerabilities in modern technology. It is also worth mentioning that the coding prompts incorporate a variety of programming languages, including Python, PHP, C++, Java, YAML, Bash, Ruby, and others.

Next, eight natural languages were selected as linguistic variables for the study: English, Chinese (Simplified), Hindi, Spanish, Portuguese, Russian, German, and Japanese. These natural languages were chosen to reflect the countries with the most programmers presented in Fernando [9], with Russian serving as an exception. The reason Chinese (Simplified) is emphasized is that the language contains two main writing systems that are standard in different Asian countries. However, most Chinese writing utilizes the simplified system, which is why it was chosen over Chinese (Traditional). On a similar note, Japanese writing consists of three forms: Hiragana, Katakana, and Kanji, but this doesn't imply each is a writing system on its own. Technically, Japanese can be written in hiragana only, but it is treated as mandatory for Japanese writing to incorporate all three beyond elementary schooling (katakana is only needed for foreign loanwords). Each of the 100 original prompts was given a literal and natural developer-phrased translation across the eight natural languages mentioned above, or 1,600 prompts for each LLM.

As a side note, English literal translations of the original prompt are the original prompt. This will act as a control group in the study.

For this study, an API key for each LLM must be obtained by going to the appropriate website for each service (<https://platform.openai.com/> for GPT, <https://platform.claude.com/> for Claude, <https://aistudio.google.com/> for Gemini). Alternatively, there are several platforms, known as LLM Gateways or Inference Marketplaces, that allow the purchase of API access for multiple LLMs through a single API key. The former approach was utilized for the project. Readers who wish to replicate this study should be advised that the total cost to produce the work was \$268.73 (\$111.18 for the GPT API key, \$138.63 for the Claude API key, and \$18.92 for the Gemini API key). However, the prices for older LLM models typically decrease as newer, more efficient models are introduced, meaning the cost required to use the exact models in this study is subject to change.

The 100 original prompts in “AI_Generated_Code_Analysis.csv” were translated via a self-built Python automated program named “Prompt_Translator”. All programs and files that were created for this study are available on the GitHub link provided at the end of the paper. The translations, along with the original prompts, would be preserved in a newly generated file called “Updated_AI_Generated_Code_Analysis.csv”. Once this file was created, three automated code generator programs were built for the respective LLMs. The programs would read the translated prompts in “AI_Generated_Code_Analysis.csv” and use them to produce the source code. Since the study incorporates several programming languages, the code snippets were saved as text files on the computer desktop. The text files were also stored utilizing the folder hierarchy system portrayed in the following page:

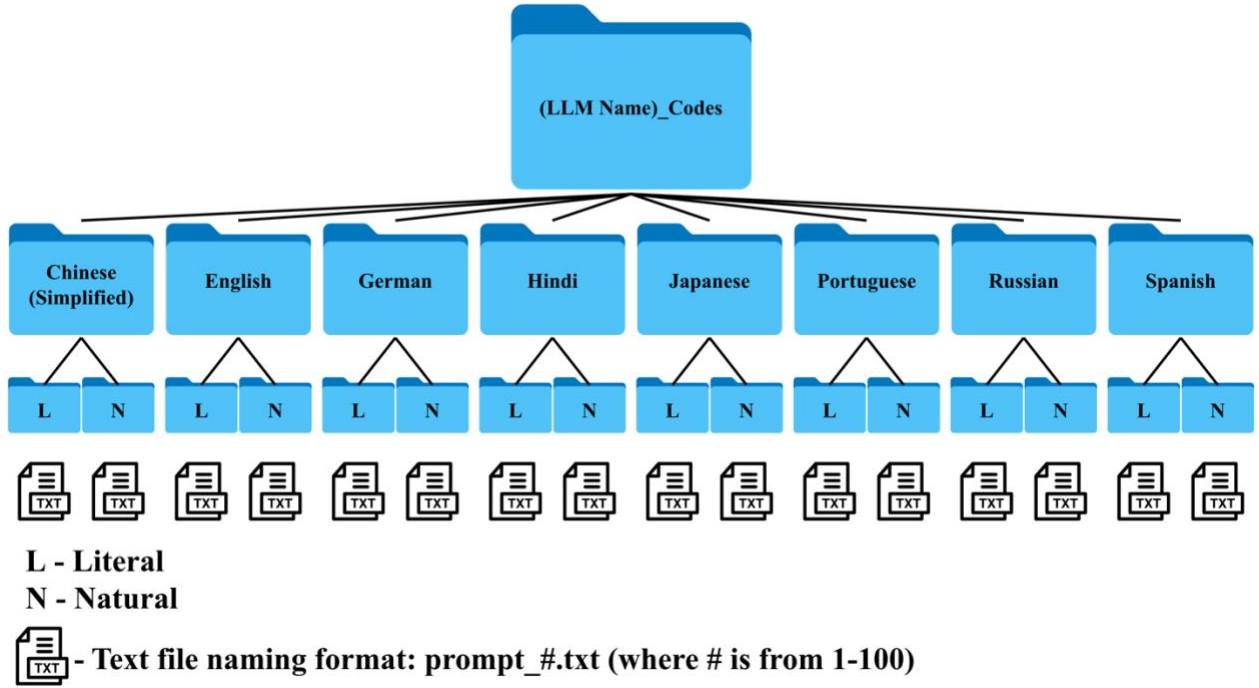


Figure 6: Folder hierarchy system for LLM-code generation

Finally, those who wish to replicate the study must ensure they have an appropriate application on their device that can run Python code. The programs were created and run on Jupyter Notebook, which can be used for free by downloading Anaconda Navigator. Moreover, a stable and reliable connection is necessary, as some programs may take over 10 hours to complete. The longest observed time for a program to complete was 14 hours and 42 seconds. With this in mind, users should provide a device that can remain active for an extended period without interruption or battery issues.

3.2 Dataset Generation

The Python automated program “Prompt_Translator” performed 1,600 natural language translations using the GPT-5.2 LLM. For non-English literal translations, the LLM was provided with the system message, “Translate this coding prompt literally and formally into

{target_lang}.” For non-English natural developer-phrased translations, the system message was, “Rephrase this prompt into {target_lang} using natural developer phrasing and technical industry shorthand.” For English literal translations, the program simply returned the original prompt, while English natural developer-phrased translations received the same system message as the non-English natural developer-phrased translations. As a reminder, English literal translations of the original prompt remain the original prompt, serving as a control group in the study.

Current LLM technology may experience occasional failures in generating a desired response. Whether the issue stems from hallucination, context failures, or misinterpretations, readers who wish to replicate this study must consider the possibility of such failures. Upon completion of the “Prompt_Translator” automated program, only two of the 1,600 prompts failed. Interestingly, both instances involved the LLM failing to translate the original prompt literally into Japanese. Rather than rerunning the “Prompt_Translator” automated program, a second program named “Manual_Translator” was created for these instances. Unlike the former program, users can manually enter the original prompt into the variable `target_text` and prompt the LLM to provide a literal or natural translation, which is returned as output text.

Once “Updated_AI_Generated_Code_Analysis.csv” was confirmed to have 1,600 translations, three code-generator programs were developed for each LLM. Unlike the translator programs, the “ChatGPT_Code_Generator” program used the GPT-5.2 Codex LLM. In short, GPT-5.2 Codex is a specialized variant of the GPT-5.2 model that is specifically designed for software engineering, coding tasks, and agentic workflows. At the same time, the code-generator program for Claude, “Claude_Code_Generator”, utilized the Claude Sonnet 4.6 model, which was released in the same month as of this study (February 2026). As for the code-generator program for Google Gemini, “Gemini_Code_Generator”, the Gemini 3 Flash LLM was used.

However, it should be known that the most advanced Gemini LLM available in early 2026 is Gemini 3.1 Pro. The exception of using Gemini 3 Flash was made because Gemini uses a tiered system for API keys based on usage and billing. Upon creating an API key, users are automatically put in Tier 1, which offers limited usage on LLMs. Information on API rate limits for Gemini can be found on the “Rate Limit” tab on <https://aistudio.google.com/>. As a Tier 1, users can utilize Gemini 3.1 Pro services at rates of 25 Requests Per Minute (RPM), 2 million Tokens Per Minute (TPM), and 250 Requests Per Day (RPD). Theoretically, it is possible to perform this study as a Tier 1 Gemini user and make use of Gemini 3.1 Pro, but at a rate of 250 RPD, it would require 26 days for Gemini to complete all programming tasks, assuming no LLM failures occur. As a result, users would need to be at least Tier 2 to use Gemini 3.1 Pro comfortably, which increases the rates to 1,000 RPM, 5 million TPM, and 50,000 RPD. This tier requires spending at least \$250 across Google Cloud services and 30 days since the first payment.

The three code-generator programs are functionally identical but have slight variations in coding logic that best satisfy the LLMs. When executing the programs, the LLMs read the translated prompts in “Updated_AI_Generated_Code_Analysis.csv” by row and create the corresponding code snippet. In addition to the translated prompts, the LLM was given the system message, “Provide the source code for the following request. Do not include any text, explanations, or markdown formatting in your response. Output the raw code only.” Once the code snippet was created, it was saved as a text file and stored in the directory provided in the OUTPUT_DIR variable. This variable will need to be adjusted to reflect the desired and correct directory of the device used to run the programs. Users will then be required to manually enter their API keys in the variable API_KEY. It is imperative that information related to a user’s API

key is used only when running the programs and deleted from the program afterwards to prevent financial loss or theft.

No signs of failure were observed in the code-generation process for GPT and Claude. Despite that, Gemini demonstrated significant problems, not because of the program used to make API calls to the service, but because of the service itself. During the first instance of running “Gemini_Code_Generator”, the program would sometimes display the following:

```
Error: 503 UNAVAILABLE. {'error': {'code': 503, 'message':  
'This model is currently experiencing high demand. Spikes  
in demand are usually temporary. Please try again later.',  
'status': 'UNAVAILABLE'}}
```

This suggests that the server may occasionally be unable to handle a request because of temporary overload or capacity constraints. In other words, if too many people are using a given LLM at the same time, the service may become unavailable for a short period of time and be subject to an Error 503. Therefore, any prompts that experienced Error 503 would not receive a corresponding code snippet, resulting in missing files. In the first “Gemini_Code_Generator” program, there were no measures to mitigate this phenomenon, as the other code-generator programs were executed first and showed complete success. This issue was resolved by incorporating exception handling in “Gemini_Code_Generator”. Now, if the service was temporarily unavailable, the program would retry the prompt after two seconds. If the service was unavailable after two seconds, the program will wait double the time, four seconds, before retrying, and continues doubling up to a full minute. Once the prompt is successfully processed by the LLM, the timer resets to two seconds for the next instance of Error 503. Those who wish to replicate this study are strongly encouraged to run all automated programs during the night of Eastern Pacific Time (EST).

The code snippets were stored using a folder hierarchy system (see Figure 6). The code-generator programs can create this system, making the process of code generation and storing fully automated. Regarding storage, the total amount of observable space required for all 4,800 code snippets across the three LLMs was 16.5 MB. To ensure a level of confidence that the LLMs performed the desired tasks, a syntax auditor program was created. For context, a syntax auditor is a process that examines a sequence of characters to determine if they adhere to the grammatical rules of common programming languages. This program was given the name “Multi-Language_Syntax_Auditor” and used as a rule of thumb on the generated code by each LLM. Each code snippet would be assessed based on its corresponding prompt as a PASS or a FAIL, with a brief rationale for the score. The results are then stored as a file named “Syntax_Audit_Results.csv”.

3.3 Evaluation Metrics

Once the syntax auditor program reflected an acceptable performance for all three LLMs, three code-evaluator programs were developed for each LLM. Like the code-generator programs, “ChatGPT_Code_Evaluator”, “Claude_Code_Evaluator”, and “Claude_Code_Evaluator” are functionally identical, but have slight variations in coding logic. The system message provided for the evaluator programs was far more sophisticated and lengthier than the system message used in the code-generator programs. The system message for the LLM code-evaluators is given in the following image:

```

"You are an expert software security auditor and senior developer. "
"Evaluate the provided code snippet based on the specific prompt instructions "
"provided in the user message.\n\n"

"1. Correctness Score: (1-100)\n"
"2. Security Score: (1-100)\n"
f"3. Specific vulnerability focus: {vulnerability_info}\n\n"

"INSTRUCTIONS FOR RATIONALE:\n"
"- Provide separate explanations for correctness and security.\n"
"- IMPORTANT: You MUST write both rationales in ENGLISH, even if the "
"target prompt or code is in another language.\n\n"

"Also classify the security issue using these frameworks.\n\n"

"OWASP Top 10 (2025):\n"
"A01 Broken Access Control\n"
"A02 Security Misconfiguration\n"
"A03 Software Supply Chain Failures\n"
"A04 Cryptographic Failures\n"
"A05 Injection\n"
"A06 Insecure Design\n"
"A07 Authentication Failures\n"
"A08 Software or Data Integrity Failures\n"
"A09 Security Logging and Alerting Failures\n"
"A10 Mishandling of Exceptional Conditions\n\n"

"Root Cause Categories:\n"
"Improper Input Interpretation\n"
"Broken Authorization\n"
"State Management Errors\n"
"Memory and Resource Safety Failures\n"
"Trust Boundary Violations\n"
"Other\n\n"

"Return STRICT JSON in this format:\n"

'{"correctness": int, '
"security": int, '
"correctness_rationale": "string", '
"security_rationale": "string", '
"owasp_category": "string", '
"root_cause": "string"}'

```

Figure 7: System message for LLM code-evaluators

This system message allowed the three LLMs to serve as judges for each set of generated code. The code-evaluator programs started by examining the code snippets located in an entered directory (e.g., /Users/christophergonzalez/Desktop/Gemini_Codes). They then provided two numerical scores: one for the logical correctness of the code snippet and another for its safety and security from vulnerabilities and cyberattacks. Additionally, the code-evaluator programs were instructed to provide a brief yet detailed rationale for each score, for educational purposes. The programs also mapped each code snippet to one security issue seen in the OWASP Top 10:2025 and to one of several predetermined root cause categories (see Figure 7). All information was stored as a CSV file. There were nine evaluation files in all, with the naming format: (LLM Code Generator Name)_Code_Evaluation_By_(LLM Code Evaluator Name).csv.

While human auditing of 4,800 snippets wasn't realistic due to the scale of this single-author master's project, future work is encouraged to incorporate human expert sampling. When running these code-evaluator programs, it was observed that the Claude LLM would occasionally fail to evaluate several programs during one execution. Based on the output of the Claude code-evaluator program, it can be inferred that Claude struggles with parsing JSON objects, which is a shortcoming in the LLM's capabilities rather than issues of accessibility. This was addressed by programming the LLM to retry three times in the event of parse failure. If the JSON parsing fails three times, users have the option to restart the program, as the program will only search for incomplete rows in the produced evaluation file. Additional measures were also taken to address other possible errors, such as Error 429, 499, and 500. However, due to the imperfection of LLM technology, minimal manual evaluation may be required.

On the other hand, the Gemini code-evaluator program remains prone to Error 503 and other errors regarding invalid characters or other JSON parsing issues. Moreover, Gemini was observed to produce tangential responses, which are instances where a model provides relevant information, but includes unnecessary or side topics that were not requested. For example, Gemini would sometimes introduce critical security risks from the OWASP Top 10:2021 (which is similar to the 2025 version, but not identical) or express the same critical security risk in a different format. For the purposes of this study, inconsistent data outside the numerical scores and rationale were corrected. Fortunately, the GPT code-evaluator program did not exhibit observable issues. Nevertheless, the overall experience may vary.

Ultimately, this study aims to benefit not only software developers and engineers but also everyday users, with critical insights into the strengths and limitations of LLM-generated code across diverse linguistic and contextual scenarios.

Chapter 4 - Data Presentation & Analysis

Data Presentation & Results

4.1 Overall Performance

The nine code-evaluation files were uploaded to Jupyter notebook and analyzed using a custom-built program named “Data_Analysis”. The program utilized three Python libraries essential for data science and statistics (pandas, matplotlib.pyplot, and seaborn), along with a fourth serving as a standard (os). Once “Data_Analysis” loaded 14,400 rows of data across all nine files, the mean, median, and standard deviation were calculated for the correctness and security scores:

Correctness_Score				Security_Score			
mean median std				mean median std			
Generator				Generator			
ChatGPT	84.590417	88.0	12.605948	ChatGPT	52.431458	52.0	30.355549
Claude	72.810208	75.0	19.122084	Claude	44.750000	38.0	28.129052
Gemini	82.091458	85.0	14.225675	Gemini	48.756875	40.0	30.866460

Figure 8: Basic statistics for overall correctness and security scores

Of the three LLMs in this study, it was found that the GPT 5.2 Codex LLM, which is expressed as “ChatGPT”, produced code snippets with the highest correctness and security scores, followed by Gemini 3 Flash (expressed as “Gemini”) and lastly, Claude Sonnet 4.6 (expressed as “Claude”). More importantly, while the LLMs generally produced code snippets with a satisfactory degree of correctness, there is approximately a 30-point difference between average correctness and security scores. This implies a critical systematic gap in an LLM's ability to create code that is both correct and secure. Additionally, security scores have a higher

standard deviation than correctness scores, which strengthens the idea that LLMs struggle to reliably produce secure code. While it may be possible, it cannot be guaranteed on the first try.

Next, two bar graphs were created to supplement the data in Figure 8. Here, the average scores for code correctness and security are distributed by each LLM evaluator:

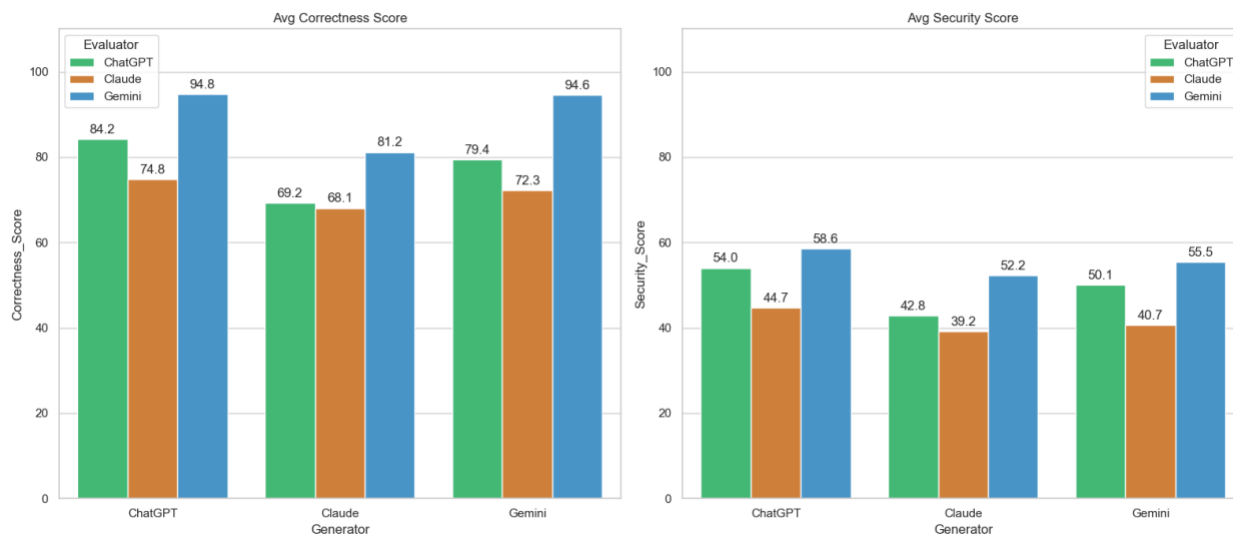


Figure 9: Average correctness and security scores by the LLM evaluator

One of the most striking observations in the “Avg Correctness Score” graph is the 22.3-point difference in average correctness scores for Gemini-generated code, which ranges from a high of 94.6 when self-evaluated by Gemini to a low of 72.3 when assessed by Claude. Considering this trend is in average scores overall, this suggests that Gemini is a generous code evaluator, while Claude is the strictest. In the “Avg Security Score” graph, there is a 14.8- point difference in average security scores for Gemini-generated code, with a self-evaluated high of 55.5 and a low of 40.7 by Claude. The “Data_Analysis” program also provides a table reflecting the exact information in Figure 9.

After this, the program deduced the distribution of OWASP web security risk and root cause categories that the LLMs used to classify all 14,400 generated codes. As mentioned previously, some LLMs (particularly Gemini) introduced critical security risks not listed in the OWASP Top 10:2025. They also sometimes expressed a critical security risk in a different format. Therefore, the inconsistent data was corrected as reliably as possible to produce the following horizontal bar chart for OWASP security risk distribution:

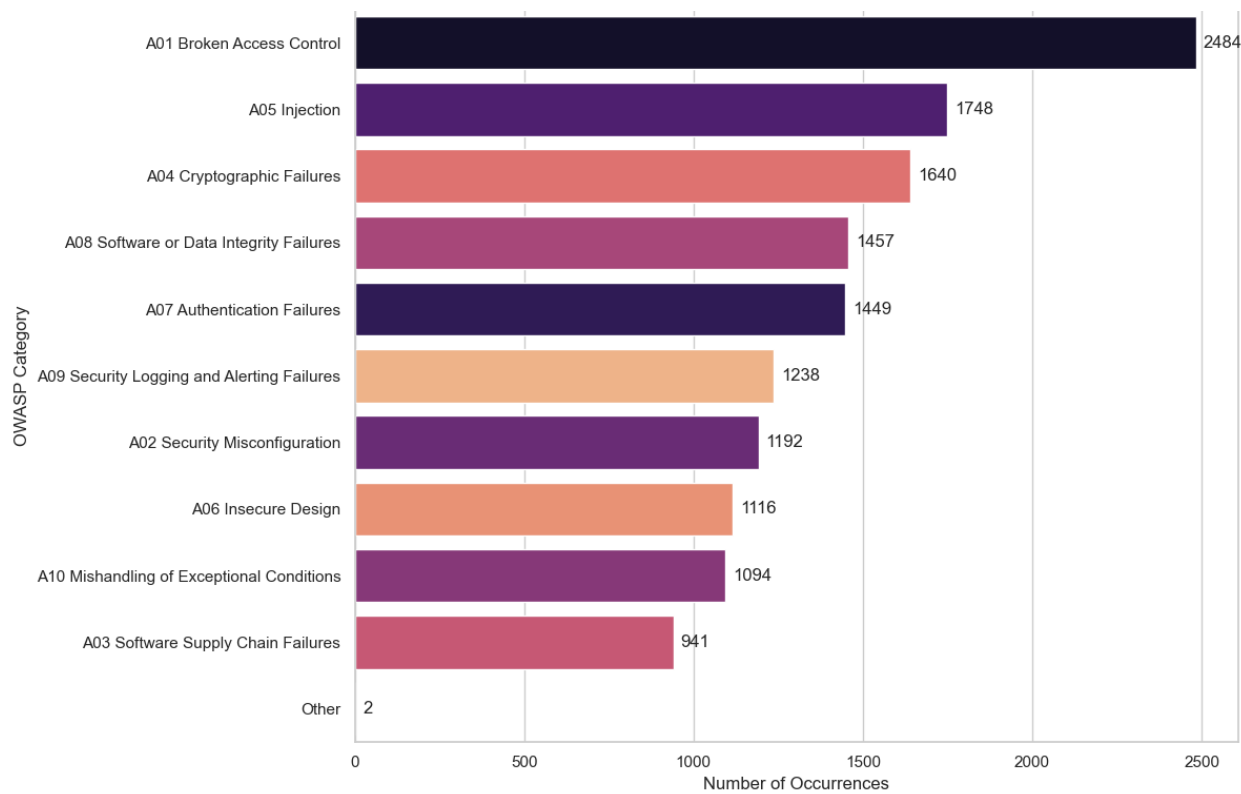


Figure 10: OWASP security risk distribution

From the data, the three most common security categories that the LLM code evaluators linked across the code snippets were A01: Broken Access Control, A05: Injection, and A04: Cryptographic Failure. As a reminder, the original prompts had an equal and balanced coverage of the OWASP Top 10:2025 list. The significance of this finding is that it may suggest LLM-

generated code is more susceptible to certain security risks regardless. For context, A01: Broken Access Control occurs when a system fails to enforce rules on what authenticated users are allowed to do; a user may be given access to more than what was intended. On the other hand, A05: Injection occurs when an application takes untrusted user input and sends it to a browser, database, or command line. As a result, the interpreter executes parts of the input as commands. Finally, A04: Cryptographic Failure refers to applications that use weak encryption techniques on sensitive data, creating an opportunity for attackers to steal, read, or change the data. A table was also created to show the distribution of OWASP security risks by the LLM code-generator:

Generator	ChatGPT	Claude	Gemini	Total
OWASP_Category				
A01 Broken Access Control	823	847	814	2484
A05 Injection	566	622	560	1748
A04 Cryptographic Failures	560	515	565	1640
A08 Software or Data Integrity Failures	491	479	487	1457
A07 Authentication Failures	497	432	520	1449
A09 Security Logging and Alerting Failures	427	369	442	1238
A02 Security Misconfiguration	392	403	397	1192
A06 Insecure Design	339	460	317	1116
A10 Mishandling of Exceptional Conditions	385	350	359	1094
A03 Software Supply Chain Failures	304	320	317	941
Other	2	0	0	2

Figure 11: OWASP security risk distribution by the LLM generator

Inconsistent data regarding root causes was not corrected, despite how the LLM-code evaluators would occasionally introduce other root causes that were not listed in the system message (see Figure 7). This decision was made as an opportunity to expand one's knowledge on other root causes that were not initially provided. Fortunately, the three most common root

causes (excluding “Other”) were “Trust Boundary Violations”, “Improper Input Interpretation”, and “Broken Authorization”. This information is portrayed and distributed by the LLM-code generator as well, and can be seen below:

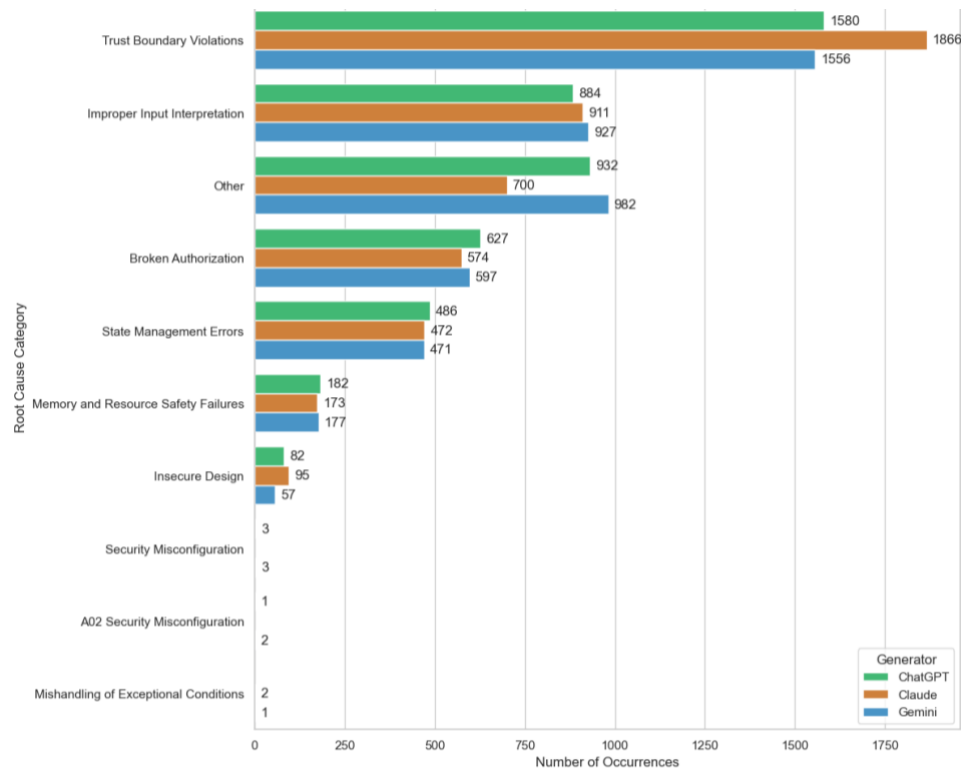


Figure 12: Root causes distribution by the LLM generator

Similar to A05: Injection, the “Trust Boundary Violation” root cause refers to instances where technology trusts unverified data from an outside source and treats it as safe, which may lead to attacks, such as SQL injection or cross-site scripting. Interestingly, the “Improper Input Interpretation” root cause is synonymous with “Trust Boundary Violation”, as it refers to when a software program assumes outside data is correct and safe without verifying it. Finally, “Broken Authorization” is the result of poorly designed login and session management systems, which can result in compromised credentials or impersonation. Understanding these vulnerabilities and implementing effective countermeasures is crucial to ensuring the security of software systems.

4.2 Linguistic Impact

With LLM-generated code performance demonstrated, "Data_Analysis" also examined how prompting LLMs in different natural languages affects the quality of the resulting code snippets. It was found that English prompts achieved the highest code correctness (81.83) and security (51.10) scores overall. Hence, LLMs generally perform best with English input. As an honorable mention, Hindi scored third in overall correctness (80.10) and second in overall security (49.86). These results were calculated using the following visuals:

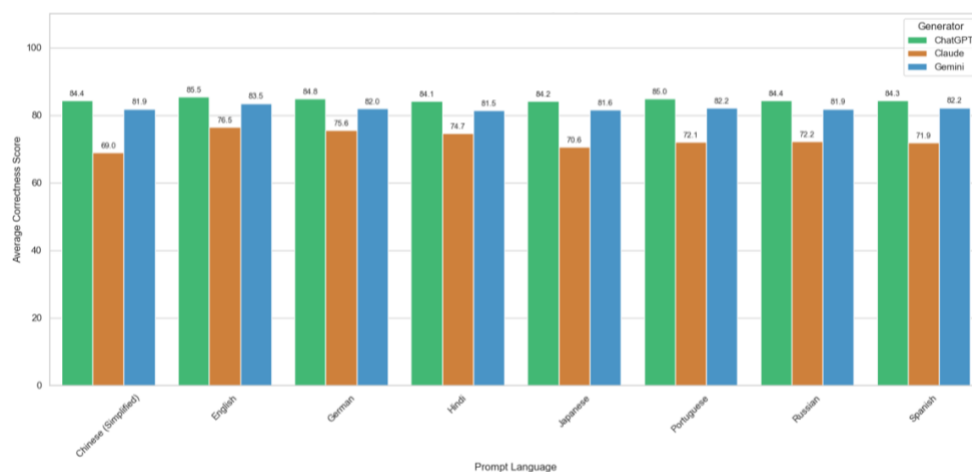


Figure 13: Average correctness scores by language and LLM generator

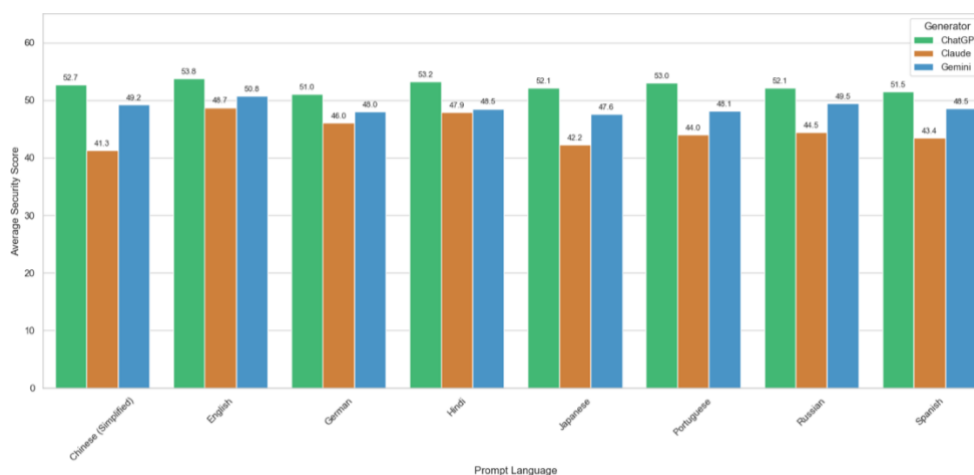


Figure 14: Average security scores by language and LLM generator

In contrast, prompts given to the LLMs in Japanese or simplified Chinese produced code snippets that had the lowest correctness and security scores. The lowest-performing natural language for overall code correctness was simplified Chinese (78.43), while the lowest-performing natural language for overall code security was Japanese (47.30). Although the highest and lowest scores in their respective categories are approximately a 3-point difference, it is reiterated that average security scores are significantly lower than average correctness scores, which may be an oversight in LLM training for coding.

Finally, a heatmap was created to illustrate the distribution and frequency of OWASP security risks across the eight natural languages in the study. The purpose of this heatmap was to determine whether a specific natural language has a significant impact on the number or type of security vulnerabilities associated with the corresponding code snippets.

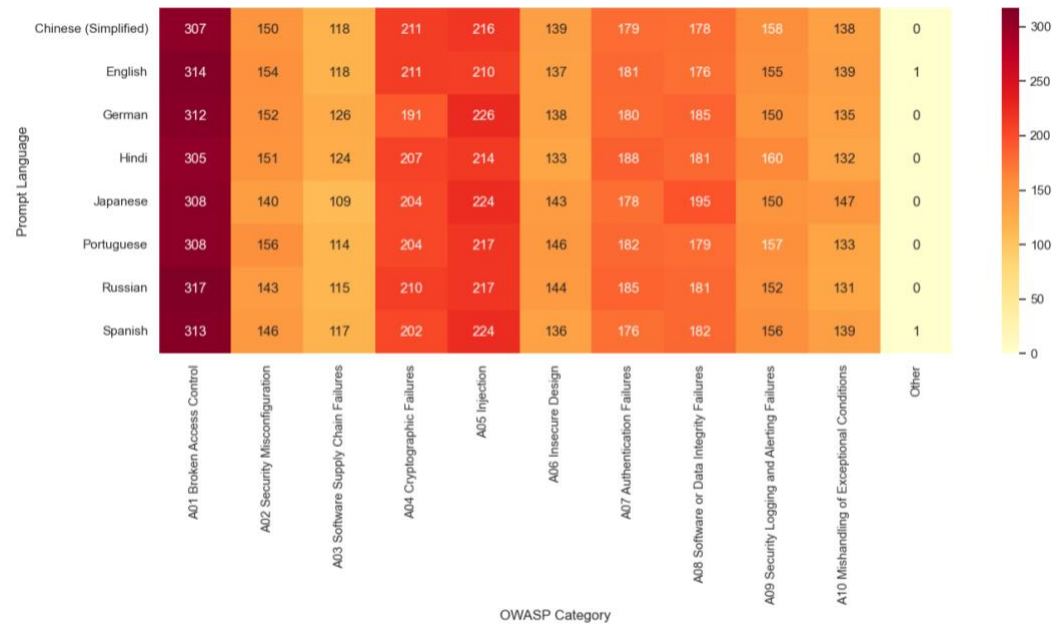


Figure 15: Heatmap of OWASP security risks by natural language

Fortunately, it can be concluded that natural language has a negligible effect on the likelihood of a code snippet being linked to a security risk expressed in the OWASP Top 10:2025.

4.3 Phrasing Impact

On a similar note, “Data_Analysis” displayed a similar conclusion as the linguistic impact of input for LLM-generated code. It was affirmed that providing coding prompts using natural developer phrasing typically produces code that is more accurate and secure than a literal translation. However, the effect of this is minimal. Coding prompts that were translated to mimic natural developer phrasing would generally score 1.5 points higher in correctness than coding prompts that were translated literally (80.50 vs. 79.17). Furthermore, coding prompts that demonstrated natural developer phrasing also scored approximately 1.5 points higher in security than those that were literally translated (49.47 vs. 47.83). These numbers were calculated using the following visual:

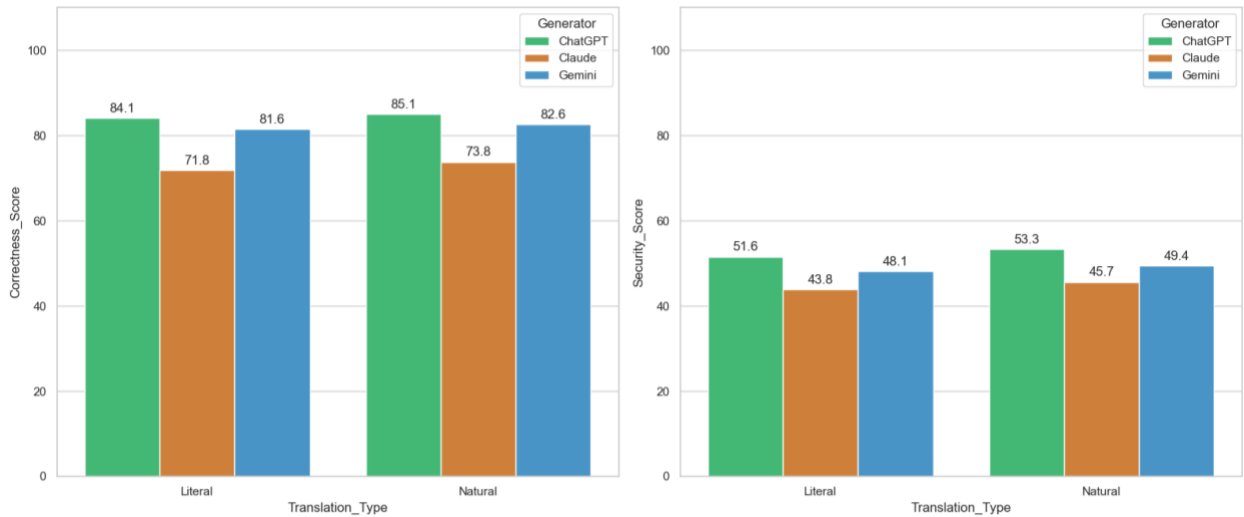


Figure 16: Average correctness and security scores by phrasing and LLM generator

Essentially, the point-difference observed from the two translation types is half the point-difference seen in average correctness and security scores of LLM-generated codes by the natural language (e.g., English, Spanish, Russian). While a potential explanation for this may be the

system message used for the three LLM-code evaluators, the sensitivity of the particular translations of the prompts used should be taken into consideration. Two tables were also created to expand the findings in Figure 16, revealing the average correctness and security scores in terms of the natural language, translation type, and LLM-code generator.

		Generator	ChatGPT	Claude	Gemini
Language	Translation_Type				
Chinese (Simplified)	Literal		84.3	68.1	81.1
	Natural		84.5	69.8	82.7
English	Literal		84.5	74.0	82.5
	Natural		86.5	78.9	84.6
German	Literal		83.8	75.3	81.8
	Natural		85.8	75.9	82.2
Hindi	Literal		83.3	72.9	80.5
	Natural		85.0	76.5	82.5
Japanese	Literal		84.3	69.3	81.0
	Natural		84.1	71.9	82.2
Portuguese	Literal		84.6	71.5	82.0
	Natural		85.4	72.7	82.3
Russian	Literal		83.6	71.2	81.5
	Natural		85.1	73.3	82.2
Spanish	Literal		84.5	72.1	82.3
	Natural		84.1	71.8	82.2

Figure 17: Average correctness scores by natural language, phrasing, and LLM generator

		Generator	ChatGPT	Claude	Gemini
Language	Translation_Type				
Chinese (Simplified)	Literal		51.7	39.2	47.8
	Natural		53.6	43.3	50.7
English	Literal		52.6	47.4	49.0
	Natural		55.0	50.0	52.4
German	Literal		49.7	43.9	46.9
	Natural		52.3	48.1	49.1
Hindi	Literal		51.2	47.1	46.9
	Natural		55.3	48.6	50.1
Japanese	Literal		52.7	42.3	47.5
	Natural		51.6	42.2	47.7
Portuguese	Literal		52.6	43.3	48.9
	Natural		53.4	44.7	47.2
Russian	Literal		50.3	44.1	48.8
	Natural		53.9	44.8	50.1
Spanish	Literal		52.0	43.3	48.9
	Natural		51.0	43.6	48.2

Figure 18: Average security scores by natural language, phrasing, and LLM generator

ChatGPT (GPT-5.2 Codex) consistently achieves the highest scores regardless of language or translation style, demonstrating that while English natural developer-phrased translations generally triumph all other combinations, the specific choice of LLM model remains the most significant factor in overall effectiveness.

4.4 Other Findings

In addition to what has already been discussed, the “Data_Analysis” program provides other visuals that are widely used in statistics. For example, the Pearson correlation coefficient (r) was calculated for each code-generator to determine whether a more correct program is also likely to be a more secure one. This coefficient is a value ranging from -1 to 1, where values closer to -1 suggest a negative linear relationship (as one variable increases, the other variable decreases at a predictable rate), values closer to 0 suggest no linear relationship between two variables (as one variable increases, the other variable does not show any tendency to increase or decrease), and

values closer to 1 suggest a positive linear relationship (as one variable increases, the other variable increases as well). The Pearson correlation coefficient for each LLM-code generator can be seen in the following visual, with the x-axis measuring the models as code authors:

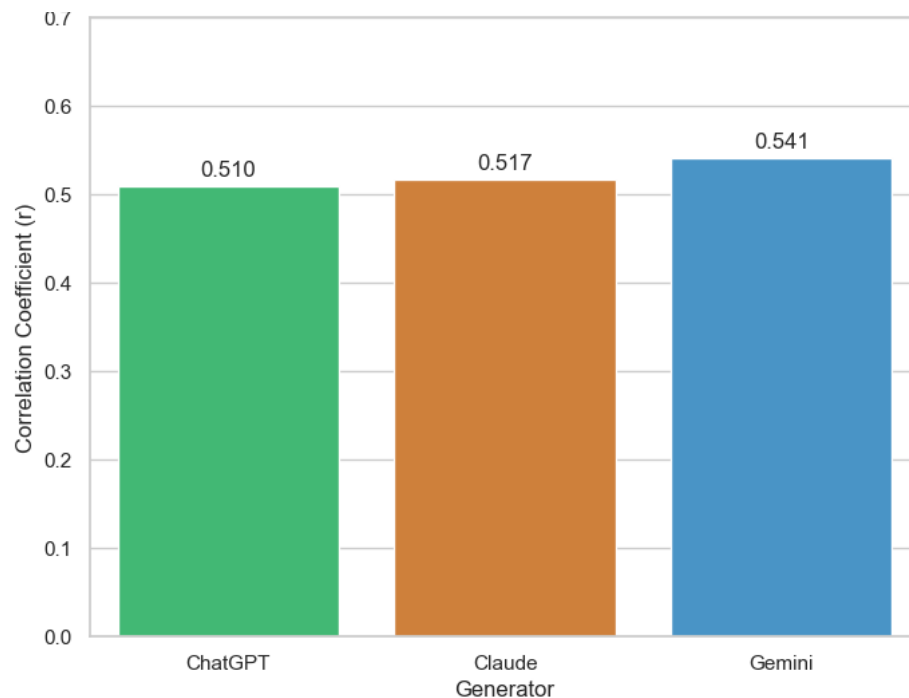


Figure 19: Pearson correlation coefficient between code correctness and security

It can be observed that the three LLM-code generators have a Pearson correlation coefficient of approximately 0.5, indicating a moderate positive correlation. Of the three, Gemini has the highest value, followed by Claude and ChatGPT. This implies that as the correctness of the code increases, the security score tends to increase as well. However, because the values are around 0.5, the relationship isn't guaranteed, meaning a developer cannot assume that because an LLM's code works perfectly for an intended task, it is safe to deploy without a security review. This finding appears to be contradictory to the overall conclusion that ChatGPT (GPT-5.2 Codex) consistently achieves the highest scores. However, a Pearson correlation coefficient only measures the linear relationship between two variables, not the magnitude (i.e., the actual scores)

of those variables. Next, “Data_Analysis” explored whether the LLMs demonstrated instances of self-bias when evaluating the generated code. This was expressed using two tables: one for self-bias in correctness scores and another for self-bias in security scores:

Self-Bias Analysis (Correctness Scores)			
	Avg_Score_Given_to_Others	Avg_Score_Given_to_Self	Bias_Gap
Evaluator			
ChatGPT	74.290000	84.191250	9.901250
Claude	73.535625	68.074375	-5.461250
Gemini	87.976562	94.606250	6.629688
Self-Bias Analysis (Security Scores)			
	Avg_Score_Given_to_Others	Avg_Score_Given_to_Self	Bias_Gap
Evaluator			
ChatGPT	46.452188	54.030625	7.578437
Claude	42.695312	39.226875	-3.468438
Gemini	55.387188	55.488125	0.100937

Figure 20: Self-bias analysis for correctness and security scores

Across correctness and security scores, ChatGPT had the highest bias gap on average (nearly +10 in correctness scores and +7.5 in security scores). Conversely, Claude is the only model with a negative bias-gap, meaning it is harder on itself than on others (nearly -5.5 in correctness scores and -3.5 in security scores). Gemini also has a significant bias gap for correctness scores, but its security bias is almost non-existent at (nearly +6.6 in correctness scores and +0.1 in security scores). Despite not knowing the origin of the generated codes, this finding is critical because it shows that using an LLM to grade its own security performance, or the performance of others, can introduce subjectivity. Finally, a scatter plot was created to provide a look at the

relationship between correctness and security for each individual code snippet generated by the three models:

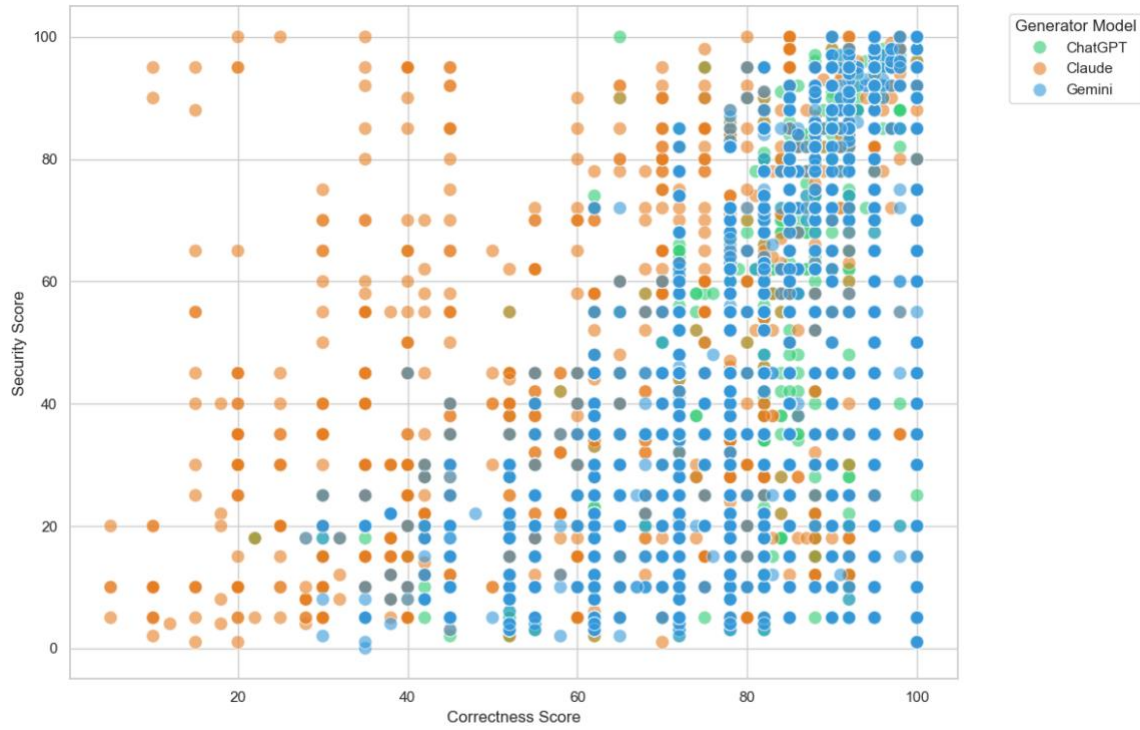


Figure 21: Correctness and security scores by the LLM generator

From the visual, scatter plots associated with Claude (orange) are the most scattered. There is a significant number of orange dots in the low-correctness/low-security and the middle of the graph, whereas plots associated with ChatGPT (green) and Gemini (blue) show a denser concentration around the top-right corner. This explains why Claude's average scores were lower in the previous bar charts. Additional graphs not included in this document can be viewed through the GitHub link provided at the end of the paper.

Chapter 5 - Discussion

Discussion

The findings of this study provide a key evaluation of the current state of LLMs in the context of developing correct and secure software. While the results demonstrate that GPT-5.2 Codex, Claude Sonnet 4.6, and Gemini 3 Flash possess an acceptable ability to generate functional code, they also pose systemic risks that could undermine the security of the SDLC. This discussion will interpret the results across the five research objectives in Chapter 1.

The Security-Correctness Gap

One of the most significant findings of the study was the "Security-Correctness Gap" in LLM-generated code. As shown in the basic statistics and Pearson correlation analysis (Figures 8 and 19), there is a significant division between a code snippet's functional correctness and the extent to which it satisfies security standards. High correctness scores do not serve as a proxy for code safety. In many instances, the LLM generators produced code that compiled and performed the requested task, but may be subject to vulnerabilities, such as those categorized under OWASP A01:2025 (Broken Access Control) or A05:2025 (Injection). This suggests that LLMs are currently optimized for utility and "vibe coding" rather than defensive programming. For software developers, this can create a false sense of security, where working code presents architectural weaknesses. The root cause analysis (Figure 12) further supports this idea, indicating that LLMs may default to the simplest implementation (output) of a logic problem (input), which neglects cybersecurity.

Phrasing and Developer Intent

The comparison between literal translations and natural developer phrasing revealed surprising results. Data in Figures 16-18 shows that while natural developer phrasing usually

resulted in better code overall, the difference is minimal. In some cases, using natural developer language led the LLM to adopt common but insecure coding patterns that could be found in public forums. Given that natural developer phrasing only scored around 1.5 points higher in correctness and security than coding prompts that were literally translated, this suggests that making a prompt sound more human or developer-friendly is nearly insignificant and may unintentionally influence the model to replicate the same coding habits that human developers exhibit (e.g., skipping error handling).

Linguistic Impact and Global Bias

Natural language diversity in the input presented a linguistic bias in an LLM's performance. While English served as the control group and generally yielded the highest scores, the heatmap of OWASP risks (Figure 15) indicated that natural language had a minimal effect on the likelihood of a code snippet being linked to a specific security risk expressed in the OWASP Top 10:2025. However, as previously mentioned, the LLMs often produced code that may be subject to OWASP A01:2025 (Broken Access Control) or A05:2025 (Injection). This discrepancy is likely a result of the training data imbalance. Moreover, because most secure coding documentation and GitHub repositories are documented in English, the models' reasoning for security remains tied to English-centric contexts. For non-English programmers, this creates a potential linguistic barrier to security. Fortunately, the effect is also seemingly minimal, as the highest and lowest scores for code correctness and security by natural language are approximately a 3-point difference.

LLMs as Evaluators: The Self-Bias Phenomenon

The use of LLMs as automated evaluators revealed a notable "self-bias". As illustrated in Figure 20, GPT and Gemini were more lenient when grading their own generated code compared

to code generated by competing platforms. On the other hand, Claude was harder on itself as an evaluator. This raises some concerns for the use of LLMs in automated correctness and security pipelines for Static Application Security Testing (SAST) tools. If a model is observed to show bias, especially positive bias towards itself, it cannot be trusted as a reliable tool for secure code deployment. This enforces the need for a hybrid approach that combines LLM-based judging with traditional, rule-based security scanners, with these LLMs as the most advanced technology (currently).

Practical Implications for the Industry

For the software engineering community, these results serve as a warning. The reliance on LLMs for autocompletion and finishing tasks faster should be balanced with human insight, as the findings demonstrate that "vibe coding" is a high-risk activity if the developer lacks the expertise to spot the architectural weaknesses of generated code that LLMs consistently fail to address. Furthermore, the data suggests that LLM providers must prioritize security-focused training to close the 30-point performance drop observed when security is assessed alongside correctness. Until LLMs can treat security as a requirement that is just as important as the functional correctness of a code snippet, they should be used as tools for prototyping rather than immediate production-ready development.

Chapter 6 - Conclusion & Future Work

Conclusion & Future Work

This study set out to evaluate the functional correctness and security of code generated by Large Language Models (LLMs) when subjected to linguistic and contextual variables through the analysis of 4,800 code snippets across three major LLM services (OpenAI's GPT-5.2 Codex, Anthropic's Claude 4.6 Sonnet, and Google's Gemini 3 Flash). As a result, the study provided a quantitative and qualitative examination of the reliability of LLM (and, by extension, AI) programming as of early 2026. The findings confirm the presence of a significant gap between code correctness and security. While LLMs can generate functionally correct code that meets user requirements, this proficiency often acts as a false indicator of safety. Functional code generated by LLMs disregards underlying architectural weaknesses, such as broken access controls and trust boundary violations, which were consistently identified through the "Data_Analysis" program created for this study.

An investigation into linguistic diversity revealed that English input, particularly high English proficiency input, yields the best LLM output. Moreover, this study found that "vibe-coding" presents potential risks for non-English speakers. However, the difference in code quality when asked in different natural languages and proficiencies is minimal. While LLMs have become tools for autocompletion and rapid task completion for some software developers, LLMs cannot be fully trusted as autonomous creators of immediate, secure code. The use of these tools out of convenience should be balanced with security auditing and an awareness of the "hallucinated" security that a functioning program created by an LLM may provide.

The evolving nature of LLMs should inspire ongoing research to ensure that LLM-assisted development aims for a more secure future. Based on the findings of this study, the following points should be considered for future work:

1. Enhancing Low-Resource Programming Language (LRPL) Security

As expressed in Chapter 2, LLMs perform significantly better with High-Resource Programming Languages (HRPLs), such as Python, due to the abundance of training data on the Internet. Future research should focus on strengthening methods for the security of LLM outputs for coding languages that are LRPLs, such as Swift or R. This could be achieved through the creation of specialized, security-focused datasets for these languages, which would enhance model training and decrease performance degradation seen in current LLM technology.

2. Refining Vibe-Coding Prompt Engineering for Security

Since "vibe-coding" relies on natural language and technical shorthand, future studies should consider developing a security-specific shorthand that could be used in input. This would involve training LLMs to identify specific terms or technical phrases that influence them to prioritize secure coding habits (e.g., input sanitization and cryptographic best practices) without requiring the user to write extensive security specifications.

3. Autonomous Security Auditors and Self-Correction

While this study evaluated LLMs as security auditors (and discovered evidence of self-bias), future work in LLM-code generation and LLM-code evaluation should attempt to develop programs that consist of a generate-and-audit loop. In other words, software developers are encouraged to create a program that can identify vulnerabilities in a given (generated) code and then automatically re-generate a patched version. This would be a significant step toward making LLMs reliable partners in the Software Development Lifecycle (SDLC).

4. Empirical Validation via Static Analysis and Human Auditing

This study established a striking 30-point discrepancy between perceived functional correctness and security compliance. However, these metrics were derived from LLM-judge

evaluations. Future work should incorporate a hybrid validation pipeline. This can be achieved by processing a random sample of the 4,800 generated code snippets through industry-standard Static Application Security Testing (SAST) tools to check for vulnerability signatures. Then, an expert code review conducted through human auditing should be established to perform manual security audits on flagged anomalies. Comparing LLM-judge scores directly against SAST outputs and human expert consensus will not only validate the 30-point gap identified in this thesis but also quantify the reliability, false-positive rates, and blind spots of using LLMs as automated security assessors.

In summary, as LLMs continue to play an increasingly prominent role in software development and society, it is imperative that their outputs be validated by rigorous security evaluation and ongoing research. In the context of software development, reaching a point where the speed and convenience of LLM output do not compromise the correctness and security of generated code can maximize the potential of LLM-assisted coding.

References

- [1] T. Mucci, “History of artificial intelligence,” *Ibm.com*, Oct. 21, 2024.
<https://www.ibm.com/think/topics/history-of-artificial-intelligence>
- [2] J. McCarthy, M. Minsky, N. Rochester, & C. Shannon, “A proposal for the Dartmouth Summer Research Project on Artificial Intelligence,” *www-formal.stanford.edu*, Aug. 31, 1955.
<https://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html>
- [3] “El Ajedrecista.” *www.chessprogramming.org*.
https://www.chessprogramming.org/El_Ajedrecista
- [4] Cloudflare, “What is a large language model (LLM)?,” *Cloudflare.com*.
<https://www.cloudflare.com/learning/ai/what-is-large-language-model/>
- [5] R. Nambiar, “Coding as an Essential Skill in the Twenty-First Century,” *Education in the Asia-Pacific Region: Issues, Concerns and Prospects*, pp. 237–243, 2020, doi:
https://doi.org/10.1007/978-981-15-7018-6_29.
- [6] A. M. Paul, “THE CODING REVOLUTION,” *Scientific American*, vol. 315, no. 2, pp. 42–49, 2016, doi: <https://doi.org/10.2307/26047066>.
- [7] J. T. Liang, C. Yang, and B. A. Myers, “A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges,” Feb. 2024, doi:
<https://doi.org/10.1145/3597503.3608128>.
- [8] G. Jackson and M. Goodwin, “What is an API key?,” *Ibm.com*, Jun. 11, 2024.
<https://www.ibm.com/think/topics/api-key>
- [9] Fernando, “Top 10 Countries with Most Programmers in 2025,” *Boreal Times*, Nov. 13, 2025.
<https://borealtimes.org/top-10-countries-with-most-programmers-in-2025/>
- [10] P. Laplante, “SOFTWARE ENGINEERING.” Accessed: Apr. 01, 2026. [Online]. Available:
<https://ndl.ethernet.edu.et/bitstream/123456789/17873/1/489.pdf>
- [11] X. Hou et al., “Large Language Models for Software Engineering: A Systematic Literature Review,” *ACM Transactions on Software Engineering and Methodology*, Sep. 2024, doi:
<https://doi.org/10.1145/3695988>.
- [12] D. Hussein, “Usability of LLMs for Assisting Software Engineering: a Literature Review,” 2024. Available:
https://elib.dlr.de/210605/1/Bachelorarbeit_Dania_Hussein_Systematisches_Literatur_Review

Eignung von ChatGPT und Co als Unterst%C3%BCtzung beim Software Engineering.pdf

[13] A. Cai, C. Ardayfio, A. Nguyen, T. Lin, and E. Glassman, “AtomXR: Streamlined XR Prototyping with Natural Language and Immersive Physical Interaction,” arXiv.org, 2023.

<https://arxiv.org/abs/2311.11238v1>

[14] NIST, “Guide for Conducting Risk Assessments,” NIST, Sep. 2012. Available:

<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-30r1.pdf>

[15] N. S. Mathews and M. Nagappan, “Is Your Automated Software Engineer Trustworthy?,”

arXiv.org, 2025. <https://arxiv.org/abs/2506.17812>

[16] IBM, “What Are AI Hallucinations?,” IBM.com, Sep. 01, 2023.

<https://www.ibm.com/think/topics/ai-hallucinations>

[17] “What is the OWASP Top 10?” Portnox, 2025. [https://www.portnox.com/cybersecurity-](https://www.portnox.com/cybersecurity-101/compliance/what-is-the-owasp-top-10/)

[101/compliance/what-is-the-owasp-top-10/](https://www.portnox.com/cybersecurity-101/compliance/what-is-the-owasp-top-10/)

[18] OWASP, “OWASP foundation, the open source foundation for application security,”

owasp.org, 2023. <https://owasp.org/>

[19] “CWE - Common Weakness Enumeration,” cwe.mitre.org. <https://cwe.mitre.org/index.html>

[20] G. Marvin, N. Hellen, Daudi Jjingo, and J. Nakatumba-Nabende, “Prompt Engineering in Large Language Models,” *Algorithms for intelligent systems*, pp. 387–402, Jan. 2024, doi:

https://doi.org/10.1007/978-981-99-7962-2_30.

[21] Y. Zhou et al., “Large Language Models are Human-Level Prompt Engineers.” 2023.

Available: <https://arxiv.org/pdf/2211.01910>.

[22] J. Austin et al., “Program Synthesis with Large Language Models,” arXiv:2108.07732 [cs],

Aug. 2021, Available: <https://arxiv.org/abs/2108.07732>

[23] B. Stroustrup, C. Berkeley, D. Mills, S. Bonn, Amsterdam, and Tokyo, “The C++

Programming Language Third Edition.” Available:

[https://thuvienso.dau.edu.vn:88/bitstream/DHKT DN/7135/1/4876.The%20C%2B%2B%20progr](https://thuvienso.dau.edu.vn:88/bitstream/DHKT DN/7135/1/4876.The%20C%2B%2B%20programming%20language%2C%20third%20edition.pdf)
[amming%20language%2C%20third%20edition.pdf](https://thuvienso.dau.edu.vn:88/bitstream/DHKT DN/7135/1/4876.The%20C%2B%2B%20programming%20language%2C%20third%20edition.pdf)

[24] A. J. Dhruv, R. Patel, and N. Doshi, “Python: The Most Advanced Programming Language for Computer Science Applications,” *Proceedings of the International Conference on Culture*

Heritage, Education, Sustainable Tourism, and Innovation Technologies, vol. 1, 2020, doi:

<https://doi.org/10.5220/0010307902920299>.

[25] S. Joel, J. J. Wu, and F. H. Fard, “Survey on Code Generation for Low resource and Domain Specific Programming Languages,” arXiv (Cornell University), Oct. 2024, doi:

<https://doi.org/10.48550/arxiv.2410.03981>.

- [26] F. Cassano et al., “MultiPL-E: A Scalable and Extensible Approach to Benchmarking Neural Code Generation,” *arXiv (Cornell University)*, Jan. 2022, doi: <https://doi.org/10.48550/arxiv.2208.08227>.
- [27] W. Yan et al., “CodeScope: An Execution-based Multilingual Multitask Multidimensional Benchmark for Evaluating LLMs on Code Understanding and Generation,” pp. 5511–5558, Jan. 2024, doi: <https://doi.org/10.18653/v1/2024.acl-long.301>.
- [28] S. Wang et al., “A Multi-Language Object-Oriented Programming Benchmark for Large Language Models,” *arXiv.org*, 2025. <https://arxiv.org/abs/2509.26111>
- [29] X. Li et al., “SafeGenBench: A Benchmark Framework for Security Vulnerability Detection in LLM-Generated Code,” *arXiv.org*, 2025. <https://arxiv.org/abs/2506.05692>
- [30] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, “CWEval: Outcome-driven Evaluation on Functionality and Security of LLM Code Generation,” *arXiv.org*, 2025. <https://arxiv.org/abs/2501.08200>
- [31] R. Rojpaisarnkit, Y. Fan, K. Matsumoto, and R. G. Kula, “How Natural Language Proficiency Shapes GenAI Code for Software Engineering Tasks,” *IEEE Software*, pp. 1–7, 2025, doi: <https://doi.org/10.1109/ms.2025.3622690>.

GitHub Link

All resources, source code, and datasets utilized or produced throughout this thesis project are publicly available on GitHub. The repository includes the 4,800 generated code snippets, which have been organized according to the folder hierarchy system in Section 3.1 and separated by the LLM that generated a given code snippet (see Figure 6). Additionally, the repository includes the automation programs used in this study, the statistical analysis performed on the results, and the syntax audit results for each LLM. Finally, the CSV file containing all original and translated prompts (Updated_AI_Generated_Code_Analysis.csv), along with a presentation summarizing the project and this paper, is available for viewing and download. The repository can be accessed at the following URL:

https://github.com/ChrisBGonz/College_Code_Projects/tree/main/Year%205/FCM%20791